

Linear Programming

ENEL445 Applied Engineering Optimisation

2025

Outline

- Linear Programs - Examples and Geometry
- Simplex Algorithm

What

Linear Programming

- Linear Programming (LP) is a special case of the general constrained optimisation problem.
- **Objective function is linear**
- **Constraints are also linear**
- m constraints and n variables

$$\begin{aligned} &\text{minimise} && f(\mathbf{x}) = \mathbf{c}^T \mathbf{x} \\ &\text{subject to} && A\mathbf{x} = \mathbf{b} \\ &&& \mathbf{x} \geq \mathbf{0} \end{aligned}$$

Linear Programming

$$\text{minimize } 2x_1 + 3x_2 - 4x_3 - 5x_4 + x_5 - 2x_6$$

$$\text{subject to } \begin{cases} x_1 - 2x_2 + 2x_5 + 7x_6 = 3 \\ 3x_2 + x_4 - x_5 + 4x_6 = 5 \\ x_3 + -3x_5 + 2x_6 = 4 \\ x_1 \geq 0, x_2 \geq 0, x_3 \geq 0, x_4 \geq 0, x_5 \geq 0, x_6 \geq 0. \end{cases}$$

Why

Linear Programming Applications

- Example applications include:
 - Manufacturing and production planning
 - Telecommunications network design
 - Portfolio selection
 - Airline crew scheduling
 - DC power flow optimisation

Linear Programming Applications

- Example applications include:
 - Manufacturing and production planning
 - Telecommunications network design
 - Portfolio selection
 - Airline crew scheduling
 - DC power flow optimisation

Objectives: The objectives typically involve minimising costs while ensuring operational efficiency and crew satisfaction. Costs include crew salaries, hotel and transportation expenses, penalties for violations of regulations, and potential revenue losses due to inefficient scheduling

Constraints: These include legal regulations (e.g., maximum duty time, rest requirements), crew qualifications (e.g., type ratings, language proficiency), operational restrictions (e.g., minimum layover times, crew base locations)

Linear Programming Applications

- In the 1990's American Airline employs more than 25,000 pilots and flight attendants to fly its fleet of over 600 aircraft.
- Crew cost is over 1.3 billion dollars/year and is second only to fuel cost.
- American Airlines spent 6000 hours of CPU time running its crew-pairing code to find an optimal solution.
- Estimated savings from implementing the solution are in excess of 20 million dollars per year.

“Few problems studied in optimisation theory have greater applications in the real world”

Example

Linear Programming

How long should Elon work in each job to maximise his weekly earnings?

- He is paid \$10 an hour tutoring
- He is paid \$7 an hour babysitting
- He wants to spend at least 3 hours, but no more than 8 hours a week tutoring
- He wants to work for no more than 20 hours per week in total

Linear Programming

How long should Elon work in each job to maximise his weekly earnings?

- He is paid \$10 an hour tutoring
- He is paid \$7 an hour babysitting
- He wants to spend at least 3 hours, but no more than 8 hours a week tutoring
- He wants to work for no more than 20 hours per week in total

Design variables

T: Tutoring hours
B: Babysitting hours

Objective function

$$E = 10T + 7B$$

E: Earnings

Constraints

$$3 \leq T \leq 8$$

$$B \geq 0$$

$$T + B \leq 20$$

Linear Programming

How long should Elon work in each job to maximise his weekly earnings?

E: Earnings

$$E = 10T + 7B$$

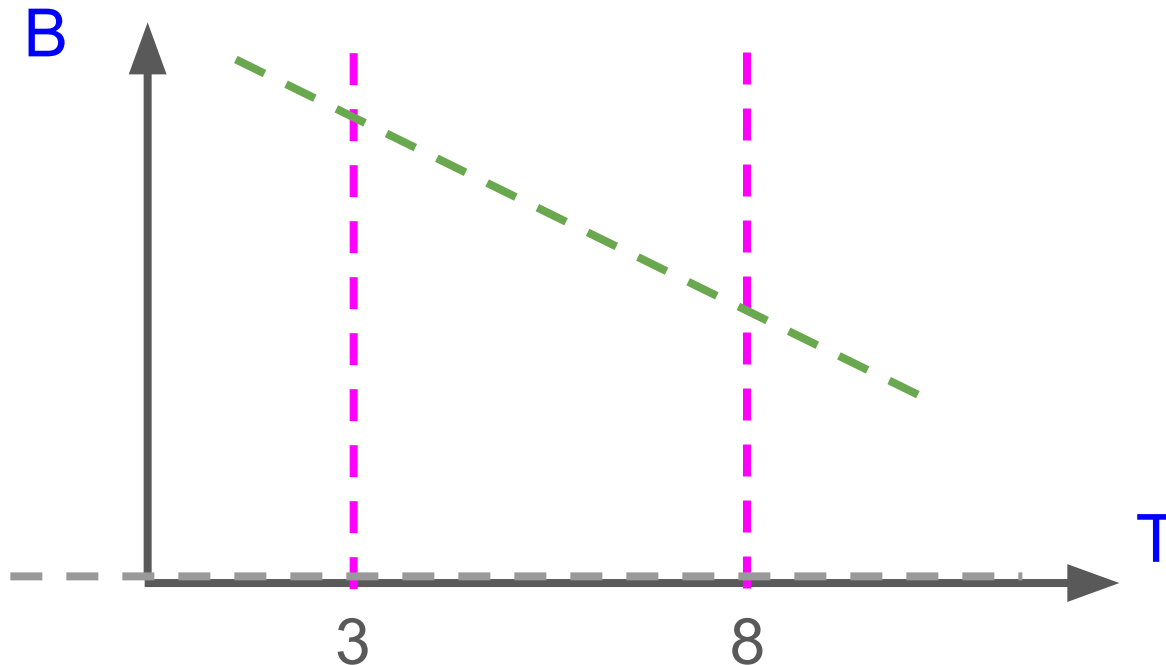
T: Tutoring hours

B: Babysitting hours

$$3 \leq T \leq 8$$

$$B \geq 0$$

$$T + B \leq 20$$



Linear Programming

How long should Elon work in each job to maximise his weekly earnings?

E: Earnings

$$E = 10T + 7B$$

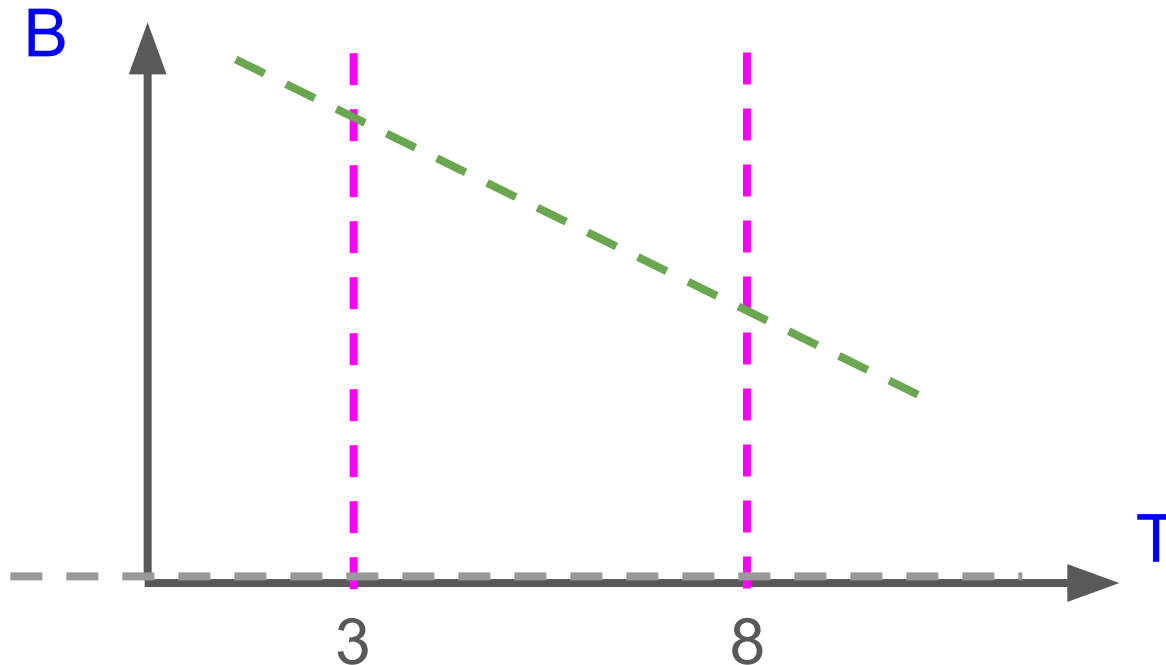
T: Tutoring hours

B: Babysitting hours

$$3 \leq T \leq 8$$

$$B \geq 0$$

$$B \leq -T + 20$$



Linear Programming

How long should Elon work in each job to maximise his weekly earnings?

E: Earnings

$$E = 10T + 7B$$

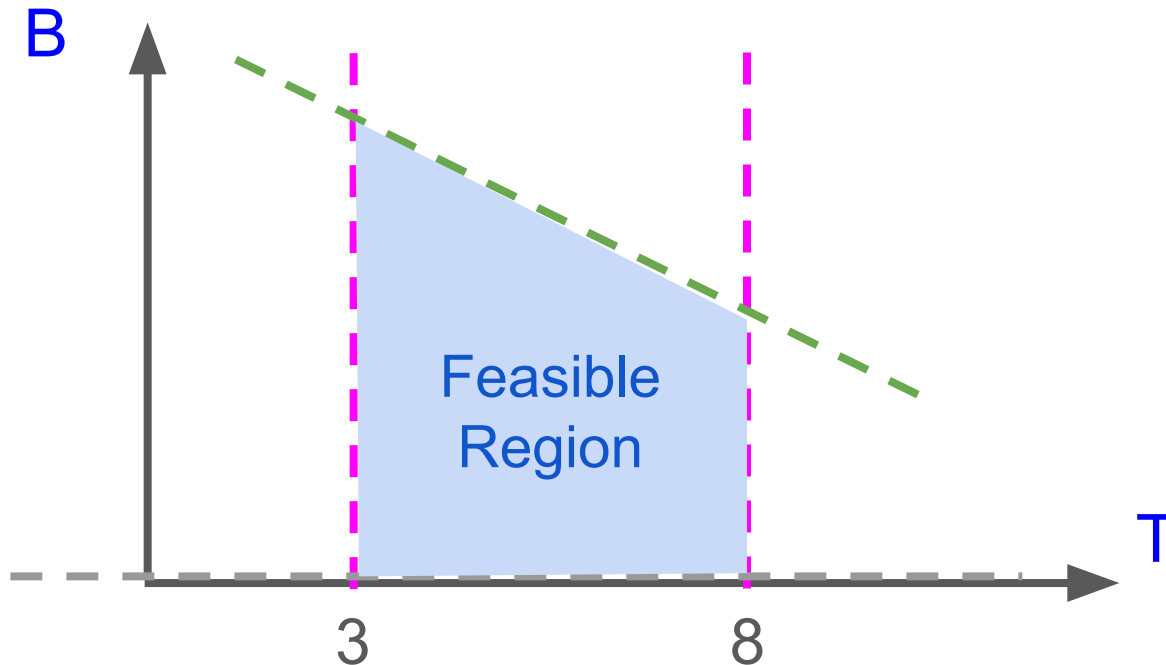
T: Tutoring hours

B: Babysitting hours

$$3 \leq T \leq 8$$

$$B \geq 0$$

$$B \leq -T + 20$$



Linear Programming

How long should Elon work in each job to maximise his weekly earnings?

T: Tutoring hours
B: Babysitting hours

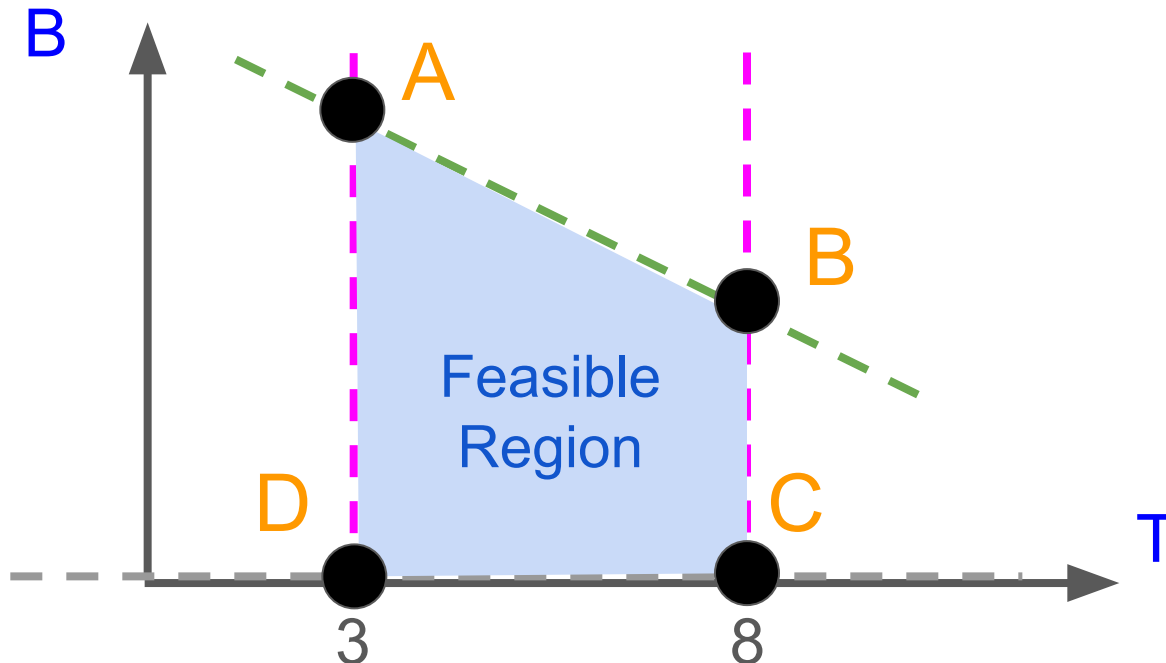
E: Earnings

$$E = 10T + 7B$$

$$3 \leq T \leq 8$$

$$B \geq 0$$

$$B \leq -T + 20$$



Linear Programming

How long should Elon work in each job to maximise his weekly earnings?

T: Tutoring hours
B: Babysitting hours

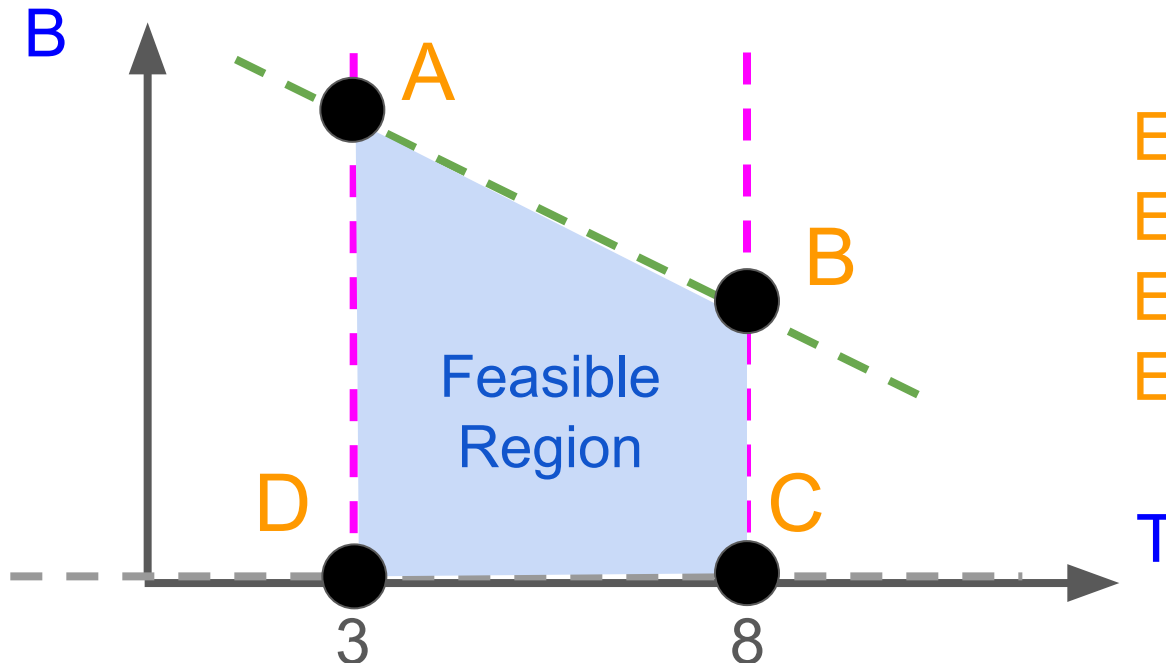
E: Earnings

$$E = 10T + 7B$$

$$3 \leq T \leq 8$$

$$B \geq 0$$

$$B \leq -T + 20$$



$$E_A(T=3, B=17) = 149$$

$$E_B(T=8, B=12) = 164$$

$$E_C(T=8, B=0) = 80$$

$$E_D(T=3, B=0) = 30$$



Linear Programming

How long should Elon work in each job to maximise his weekly earnings?

T: Tutoring hours
B: Babysitting hours

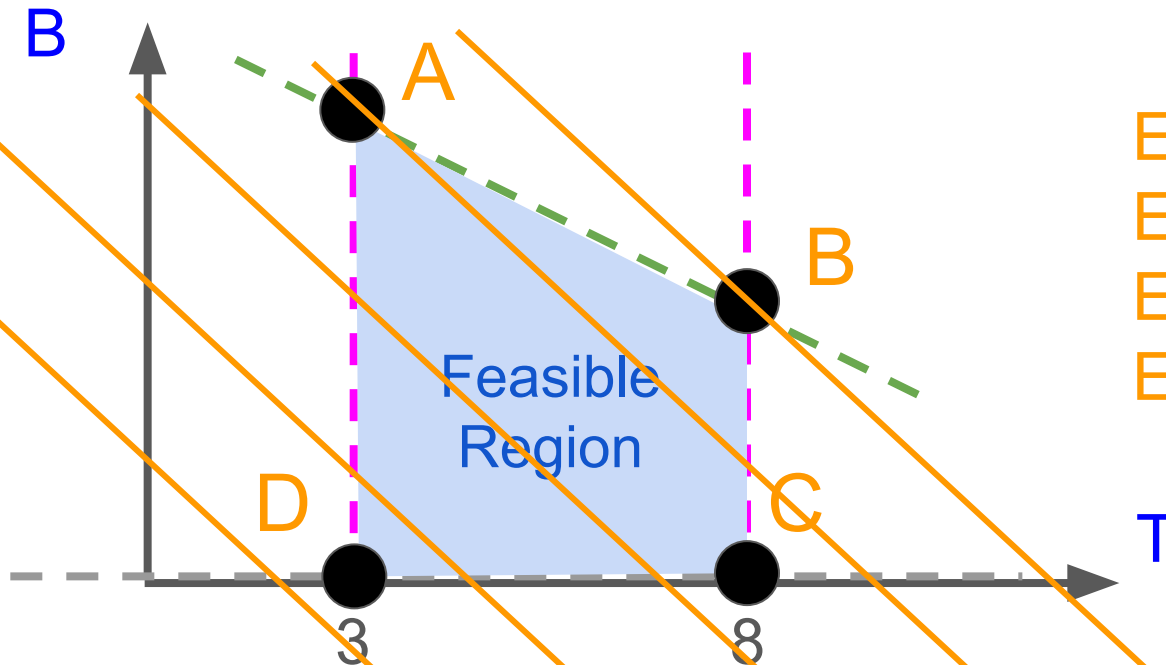
E: Earnings

$$E = 10T + 7B$$

$$3 \leq T \leq 8$$

$$B \geq 0$$

$$B \leq -T + 20$$



$$E_A(T=3, B=17) = 149$$

$$E_B(T=8, B=12) = 164$$

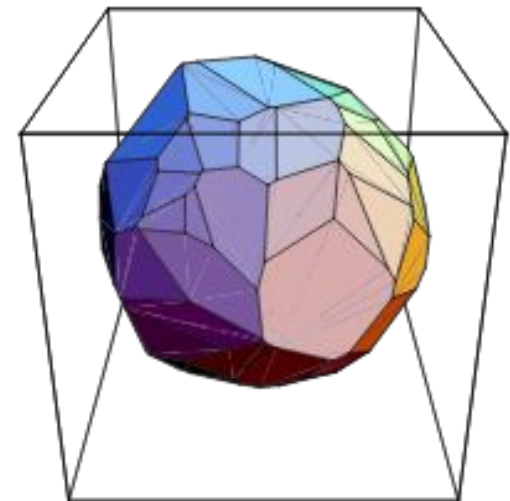
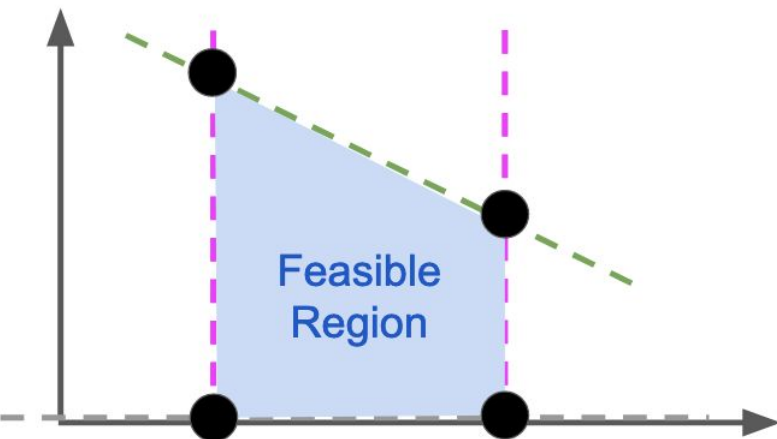
$$E_C(T=8, B=0) = 80$$

$$E_D(T=3, B=0) = 30$$



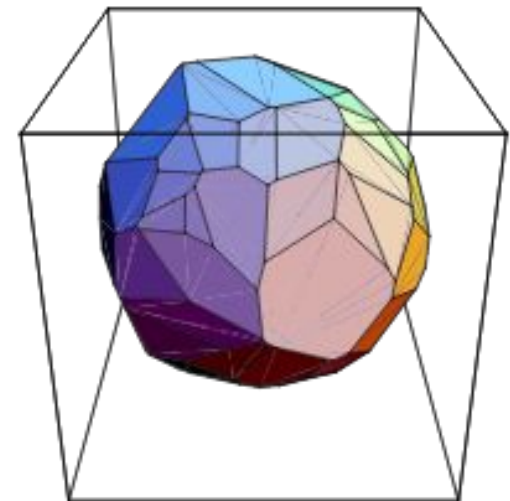
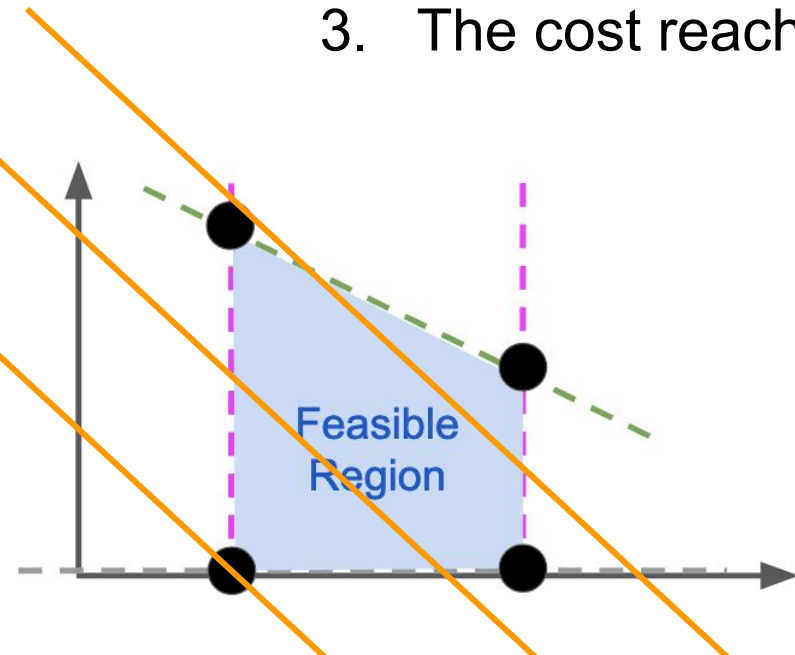
The Geometry

- **Equality constraints** give lines, planes, and hyperplanes.
- **Inequality constraints** divide space into a **halfspace** in which the constraint is satisfied and another in which it is not.
- **Feasible set** is the intersection of all halfspaces, this is called a **polyhedron**. If it's bounded and nonempty it's a **polytope**
- The problem in linear programming is to find the point that lies in the feasible set **and** minimises the cost.



The Geometry

- The optimal solution occurs at a corner of the feasible set. This is guaranteed by geometry because the constraints are linear.
- Every linear programming problem falls into one of three possible categories:
 1. The feasible set is empty
 2. The cost function is unbounded on the feasible set
 3. The cost reaches its minimum on the feasible set



How

The Simplex Algorithm

Derivation Not Examinable!
Too tedious!

Simplex Algorithm

- Top 10 algorithm of the 20th century
- Invented by George Dantzig in 1947



1946: The Metropolis Algorithm
1947: Simplex Method
1950: Krylov Subspace Method
1951: The Decompositional Approach to Matrix Computations
1957: The Fortran Optimizing Compiler
1959: QR Algorithm
1962: Quicksort
1965: Fast Fourier Transform
1977: Integer Relation Detection
1987: Fast Multipole Method

<https://pi.math.cornell.edu/~ajt/presentations/TopTenAlgorithms.pdf>

<https://algorithmsun.wordpress.com/4-top-10-algorithms-of-the-20th-century-and-algorithms-in-the-news/>

George Dantzig

https://en.wikipedia.org/wiki/George_Dantzig

In 1939, a misunderstanding brought about surprising results. Near the beginning of a class, Professor Neyman wrote two problems on the blackboard. **Dantzig arrived late and assumed that they were a homework assignment.**

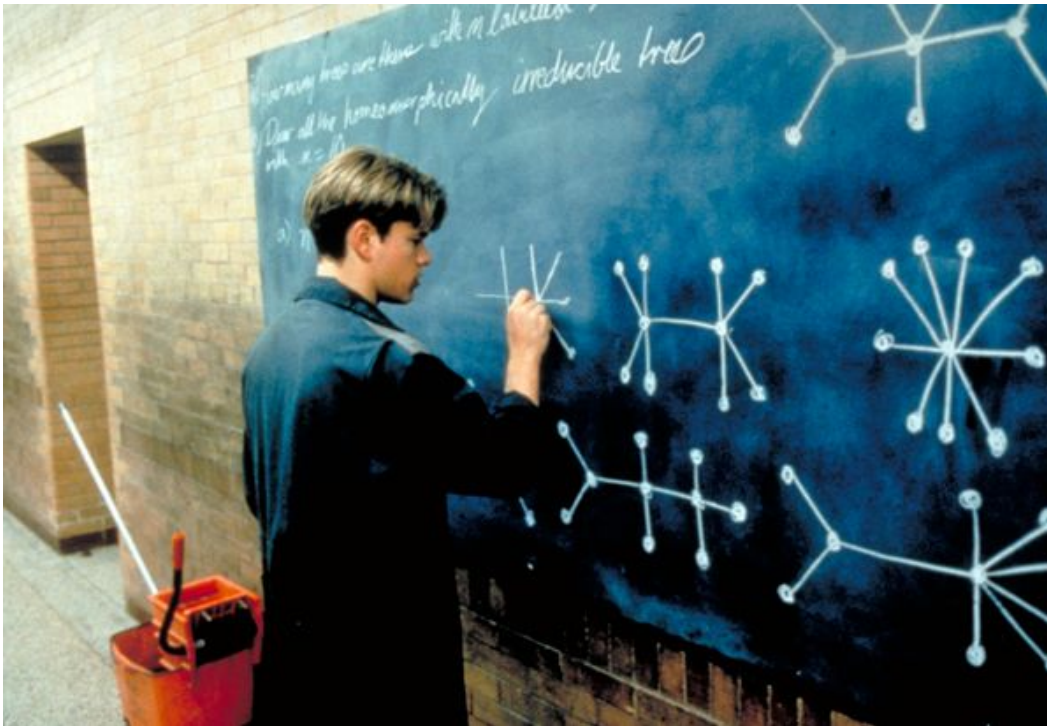
According to Dantzig, they “seemed to be a little harder than usual,” but a few days later he handed in completed solutions for both problems, still believing that they were an assignment that was overdue. Six weeks later, an excited Neyman eagerly told him that the "homework" problems he had solved were two of the most famous unsolved problems in statistics, and that he had prepared one of Dantzig's solutions for publication in a mathematical journal [1].

[1] Dantzig, George (1940). "On the non-existence of tests of "Student's" hypothesis having power functions independent of σ ". *The Annals of Mathematical Statistics*. 11(2): 186–192.

George Dantzig

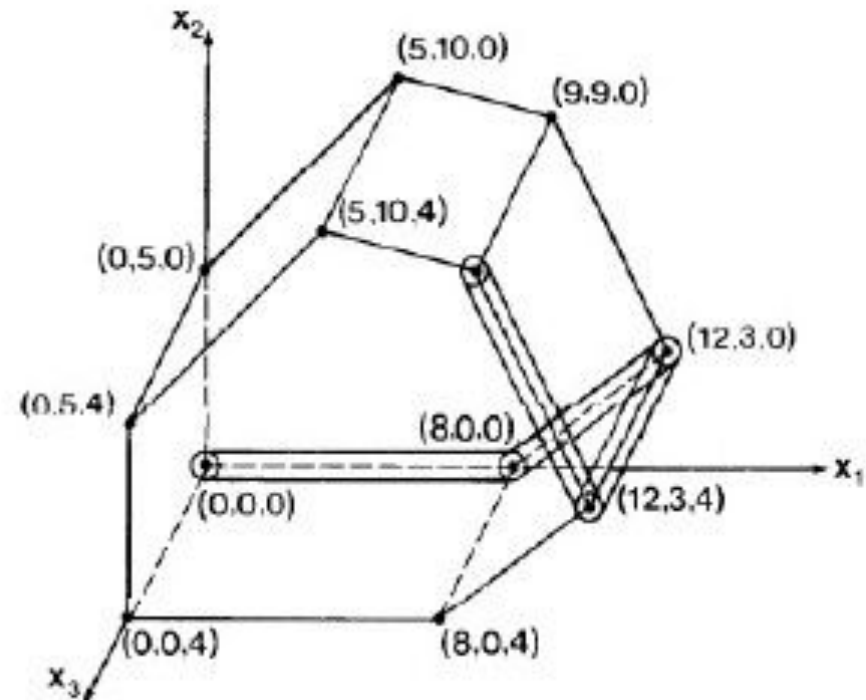
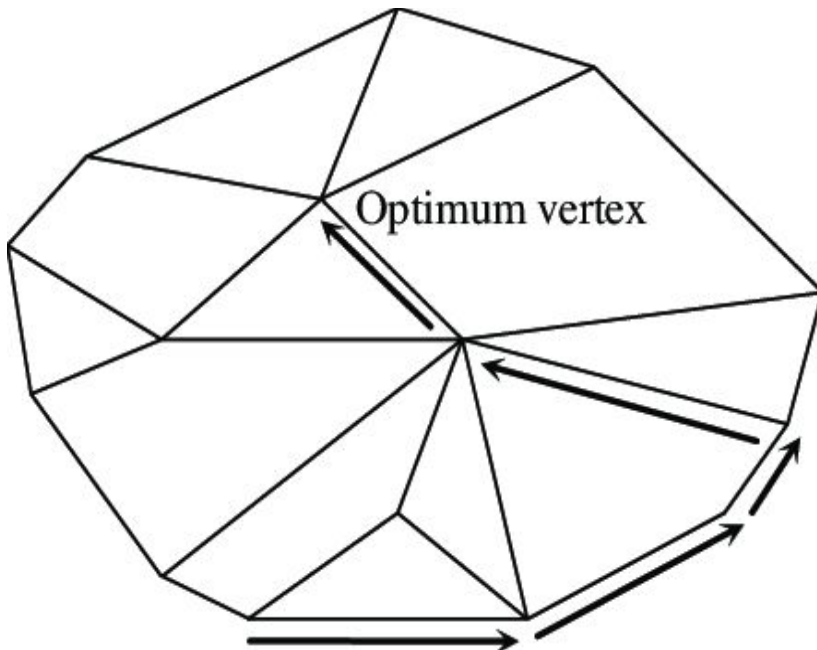
https://en.wikipedia.org/wiki/George_Dantzig

This story began to spread and was used as a motivational lesson demonstrating the power of positive thinking. Over time, some facts were altered, but the basic story persisted in the form of an urban legend and as an introductory scene in the movie *Good Will Hunting*.



Simplex Algorithm

- Geometrically, travel along the vertex (corner) of polytope in a direction that always decrease the objective function
 - **Phase I:** Locate a vertex of the feasible set.
 - **Phase II:** Move from vertex to vertex along the edges of the feasible set - always go along an edge that is guaranteed to decrease the cost.



Simplex Algorithm

1. Convert to **standard form**.
2. Gaussian eliminate until you get m solutions, Set the rest $n - m$ variables to 0. This is a **basic solution** - you are now on a vertex.
3. Calculate the **reduced-cost coefficients**, r_k .
Get the index $k = q$, corresponding to the minimum r_k .
4. Calculate the ratios b_i/y_{iq} $i = 1, \dots, m$
Get the index $i = p$, corresponding to minimum of those ratios.
5. Sub $x_q = b_p/y_{pq}$ in, reducing x_p to zero. This is **pivoting** - you are now on another vertex with a smaller objective function value.
6. Goto step 3, repeat until all r are positive - You are now at the vertex with the smallest objective value - the optimal solution!

Simplex Algorithm

1. Convert to **standard form**.
2. Gaussian eliminate until you get m solutions, Set the rest $n - m$ variables to 0. This is a **basic solution** - you are now on a vertex.
3. Calculate the **reduced-cost coefficients**, r_k .
Get the index $k = q$, corresponding to the minimum r_k .
4. Calculate the ratios b_i/y_{iq} $i = 1, \dots, m$
Get the index $i = p$, corresponding to minimum of those ratios.
5. Sub $x_q = b_p/y_{pq}$ in, reducing x_p to zero. This is **pivoting** - you are now on another vertex with a smaller objective function value.
6. Goto step 3, repeat until all r are positive - You are now at the vertex with the smallest objective value - the optimal solution!

Standard Form

- LP problem of the form below is said to be in **standard form**

$$\begin{array}{ll}\text{minimise} & f(\mathbf{x}) = \mathbf{c}^T \mathbf{x} \\ \text{subject to} & A\mathbf{x} = \mathbf{b} \\ & \mathbf{x} \geq \mathbf{0}\end{array}$$

- An example of non-standard form is

$$\begin{array}{ll}\text{maximise} & f(\mathbf{x}) = \mathbf{c}^T \mathbf{x} \\ \text{subject to} & A\mathbf{x} \leq \mathbf{b} \\ & \mathbf{x} \geq \mathbf{0}\end{array}$$

Standard Form - Slack and Surplus Variables

$$3x_1 + 5x_2 - 6x_3 \leq 7$$

$$5x_1 - 6x_2 - 4x_3 \geq 3$$

Standard Form - Slack and Surplus Variables

$$-5 \leq 3x_1 + 5x_2 - 6x_3 \leq 7$$

$$|3x_1 - x_2| \leq 10$$

Standard Form - Free Variables

- **Free variables** - variables not constrained to be ≥ 0
- There are two ways for treating free variables:
 - Introduce two other variables that are not free
 - Get rid of free variables by Gaussian eliminations

Standard Form - Example

maximize $3x_1 - 2x_2 + 4x_3$

subject to $7x_1 + 5x_2 - x_3 \leq 9,$

$$2x_1 + 2x_2 + 3x_3 = 5,$$

$$5x_1 + 6x_2 - 8x_3 \geq 10,$$

$$x_1 \geq 0 \quad x_3 \geq 0,$$

x_2 free.

Tableau

minimize $2x_1 + 3x_2 - 4x_3 - 5x_4 + x_5 - 2x_6$

$$\text{subject to } \begin{cases} x_1 - 2x_2 + 2x_5 + 7x_6 = 3 \\ 3x_2 + x_4 - x_5 + 4x_6 = 5 \\ x_3 + -3x_5 + 2x_6 = 4 \\ x_1 \geq 0, x_2 \geq 0, x_3 \geq 0, x_4 \geq 0, x_5 \geq 0, x_6 \geq 0. \end{cases}$$

$$\mathbf{c}^T: \begin{array}{ccccccc} x_1 & x_2 & x_3 & x_4 & x_5 & x_6 & \mathbf{b} \\ \left[\begin{array}{ccccccc} 1 & -2 & 0 & 0 & 2 & 7 & 3 \\ 0 & 3 & 0 & 1 & -1 & 4 & 5 \\ 0 & 0 & 1 & 0 & -3 & 2 & 4 \\ 2 & 3 & -4 & -5 & 1 & -2 & 0 \end{array} \right] \end{array} \quad \text{Tableau}$$

Simplex Algorithm

1. Convert to **standard form**.
2. Gaussian eliminate until you get m solutions, Set the rest $n - m$ variables to 0. This is a **basic solution** - you are now on a vertex.
3. Calculate the **reduced-cost coefficients**, r_k .
Get the index $k = q$, corresponding to the minimum r_k .
4. Calculate the ratios b_i/y_{iq} $i = 1, \dots, m$
Get the index $i = p$, corresponding to minimum of those ratios.
5. Sub $x_q = b_p/y_{pq}$ in, reducing x_p to zero. This is **pivoting** - you are now on another vertex with a smaller objective function value.
6. Goto step 3, repeat until all r are positive - You are now at the vertex with the smallest objective value - the optimal solution!

Simplex Algorithm

1. Convert to **standard form**.
2. Gaussian eliminate until you get m solutions, Set the rest $n - m$ variables to 0. This is a **basic solution** - you are now on a vertex.
3. Calculate the **reduced-cost coefficients**, r_k .
Get the index $k = q$, corresponding to the minimum r_k .
4. Calculate the ratios b_i/y_{iq} $i = 1, \dots, m$
Get the index $i = p$, corresponding to minimum of those ratios.
5. Sub $x_q = b_p/y_{pq}$ in, reducing x_p to zero. This is **pivoting** - you are now on another vertex with a smaller objective function value.
6. Goto step 3, repeat until all r are positive - You are now at the vertex with the smallest objective value - the optimal solution!

Basic Solution

- Perform Gaussian elimination to get m solutions, then set the rest $n - m$ variables to 0. This is a **basic solution**

Simplex Algorithm

1. Convert to **standard form**.
2. Gaussian eliminate until you get m solutions, Set the rest $n - m$ variables to 0. This is a **basic solution** - you are now on a vertex.
3. Calculate the **reduced-cost coefficients**, r_k .
Get the index $k = q$, corresponding to the minimum r_k .
4. Calculate the ratios b_i/y_{iq} $i = 1, \dots, m$
Get the index $i = p$, corresponding to minimum of those ratios.
5. Sub $x_q = b_p/y_{pq}$ in, reducing x_p to zero. This is **pivoting** - you are now on another vertex with a smaller objective function value.
6. Goto step 3, repeat until all r are positive - You are now at the vertex with the smallest objective value - the optimal solution!

Reduced-cost Coefficients

- We want to now move to another vertex, where the solution may not necessarily be basic.
- Write the first m variables in terms of the $n - m$ as

$$x_1 = b_1 - \sum_{j=m+1}^n y_{1j} x_j$$

$$x_2 = b_2 - \sum_{j=m+1}^n y_{2j} x_j$$

$\vdots$

$$x_m = b_m - \sum_{j=m+1}^n y_{mj} x_j$$

Reduced-cost Coefficients

- Substituting the system on the previous slide back into the objective function, we obtain

$$\begin{aligned} f &= \mathbf{c}^T \mathbf{x} \\ &= c_1 x_1 + \cdots + c_m x_m + c_{m+1} x_{m+1} + \cdots + c_n x_n \\ &= f_b + (c_{m+1} - w_{m+1}) x_{m+1} + \cdots + (c_n - w_n) x_n \\ &= f_b + r_{m+1} x_{m+1} + \cdots + r_n x_n \end{aligned}$$

where

$$f_b = c_1 b_1 + \cdots + c_m b_m$$

$$r_k = c_k - w_k \qquad w_k = \sum_{i=1}^m c_i y_{ik}$$

And r_k are known as the **reduced-cost coefficients**

Reduced-cost Coefficients

- f can be decreased by increasing the value of any variable for which $r_k < 0$

$$f = f_b + r_{m+1} x_{m+1} + \cdots + r_n x_n$$

- In practice it is common to choose the smallest (most negative) reduced-cost coefficient r_k so pick the index q , corresponding to the minimum r_k and increase x_q

$$q = \operatorname{argmin}_k r_k$$

Simplex Algorithm

1. Convert to **standard form**.
2. Gaussian eliminate until you get m solutions, Set the rest $n - m$ variables to 0. This is a **basic solution** - you are now on a vertex.
3. Calculate the **reduced-cost coefficients**, r_k .
Get the index $k = q$, corresponding to the minimum r_k .
4. Calculate the ratios b_i/y_{iq} $i = 1, \dots, m$
Get the index $i = p$, corresponding to minimum of those ratios.
5. Sub $x_q = b_p/y_{pq}$ in, reducing x_p to zero. This is **pivoting** - you are now on another vertex with a smaller objective function value.
6. Goto step 3, repeat until all r are positive - You are now at the vertex with the smallest objective value - the optimal solution!

Simplex Algorithm

1. Convert to **standard form**.
2. Gaussian eliminate until you get m solutions, Set the rest $n - m$ variables to 0. This is a **basic solution** - you are now on a vertex.
3. Calculate the **reduced-cost coefficients**, r_k .
Get the index $k = q$, corresponding to the minimum r_k .
4. Calculate the ratios b_i/y_{iq} $i = 1, \dots, m$
Get the index $i = p$, corresponding to minimum of those ratios.
5. Sub $x_q = b_p/y_{pq}$ in, reducing x_p to zero. This is **pivoting** - you are now on another vertex with a smaller objective function value.
6. Goto step 3, repeat until all r are positive - You are now at the vertex with the smallest objective value - the optimal solution!

Pivoting

- Question: How much can x_q be increased?
 x_q cannot change arbitrarily. Need to satisfy $Ax=b$ and $x \geq 0$

- For example consider variable $x_1 = b_1 - \sum_{j=m+1}^n y_{1j}x_j$

- x_q can only increase until $x_1 = 0$

Giving us an increase of $\frac{b_1}{y_{1q}}$

- And similarly for all variables. So search for index p corresponding to the smallest of those ratios,

$$p = \operatorname{argmin}_k \frac{b_k}{y_{kq}}$$

- And set $x_q = \frac{b_p}{y_{pq}}$

Pivoting

Simplex Algorithm

1. Convert to **standard form**.
2. Gaussian eliminate until you get m solutions, Set the rest $n - m$ variables to 0. This is a **basic solution** - you are now on a vertex.
3. Calculate the **reduced-cost coefficients**, r_k .
Get the index $k = q$, corresponding to the minimum r_k .
4. Calculate the ratios b_i/y_{iq} $i = 1, \dots, m$
Get the index $i = p$, corresponding to minimum of those ratios.
5. Sub $x_q = b_p/y_{pq}$ in, reducing x_p to zero. This is **pivoting** - you are now on another vertex with a smaller objective function value.
6. Goto step 3, repeat until all r are positive - You are now at the vertex with the smallest objective value - the optimal solution!

Simplex Algorithm and Beyond

- In practice quite fast - typically $O(m)$ pivots.
In worst case it is exponential.
- For a long time it wasn't known if there was a polynomial time algorithm until **Khachiyan's Algorithm**
 - Exterior-point method
 - “Mathematical Sputnik of 1979”

The Great Mathematical Sputnik of 1979

Eugene L. Lawler

University of California, Berkeley

- **Karmakar's Method (1984)**
 - Interior-point method
 - Competes viably with the simplex algorithm on real-world problems.

Simplex Method

<https://docs.scipy.org/doc/scipy/reference/optimize.linprog-simplex.html>



1.13.0 (stable) ▾

Installing User Guide API reference Building from source Development Release notes

scipy.cluster
scipy.constants
scipy.datasets
scipy.fft
scipy.fftpack
scipy.integrate
scipy.interpolate
scipy.io
scipy.linalg
scipy.misc
scipy.ndimage
scipy.odr
scipy.optimize
scipy.signal
scipy.sparse
scipy.spatial
scipy.special
scipy.stats

🏠 > SciPy API > Optimization and root finding (**scipy.optimize**) > linprog(meth...

linprog(method='simplex')

`scipy.optimize.linprog(c, A_ub=None, b_ub=None, A_eq=None, b_eq=None, bounds=(0, None), method='highs', callback=None, options=None, x0=None, integrality=None)`

Linear programming: minimize a linear objective function subject to linear equality and inequality constraints using the tableau-based simplex method.

❗ **Deprecated since version 1.9.0:** `method='simplex'` will be removed in SciPy 1.11.0. It is replaced by `method='highs'` because the latter is faster and more robust.

Linear programming solves problems of the following form:

$$\begin{aligned} \min_x \quad & c^T x \\ \text{such that} \quad & A_{ub} x \leq b_{ub}, \\ & A_{eq} x = b_{eq}, \\ & l \leq x \leq u, \end{aligned}$$

where x is a vector of decision variables; c , b_{ub} , b_{eq} , l , and u are vectors; and A_{ub} and A_{eq}

Simplex Method (with HiGHS)

<https://docs.scipy.org/doc/scipy/reference/optimize.linprog-highs.html#optimize-linprog-highs>



1.13.0 (stable) ▾

Installing User Guide API reference Building from source Development Release notes

scipy.cluster
scipy.constants
scipy.datasets
scipy.fft
scipy.fftpack
scipy.integrate
scipy.interpolate
scipy.io
scipy.linalg
scipy.misc
scipy.ndimage
scipy.odr
scipy.optimize
scipy.signal
scipy.sparse
scipy.spatial
scipy.special

🏠 > SciPy API > Optimization and root finding (`scipy.optimize`) > `linprog(meth...`

linprog(method='highs')

`scipy.optimize.linprog(c, A_ub=None, b_ub=None, A_eq=None, b_eq=None, bounds=(0, None), method='highs', callback=None, options=None, x0=None, integrality=None)`

Linear programming: minimize a linear objective function subject to linear equality and inequality constraints using one of the HiGHS solvers.

Linear programming solves problems of the following form:

$$\begin{aligned} \min_x \quad & c^T x \\ \text{such that} \quad & A_{ub} x \leq b_{ub}, \\ & A_{eq} x = b_{eq}, \\ & l \leq x \leq u, \end{aligned}$$

where x is a vector of decision variables; c , b_{ub} , b_{eq} , l , and u are vectors; and A_{ub} and A_{eq} are matrices.

HiGHS

Linear optimization software

HiGHS

Get started

Documentation

Workshop 2024

Newsletter

Background

HiGHS team

HiGHS - high performance software for linear optimization

Open source serial and parallel solvers for large-scale
sparse linear programming (LP),
mixed-integer programming (MIP), and quadratic programming (QP) models

Download

HiGHS

<https://github.com/ERGO-Code/HiGHS>

HiGHS is based on the high performance dual revised simplex implementation (HSOL) and its parallel variant (PAMI) developed by Qi Huangfu. Features such as presolve, crash and advanced basis start have been added by Julian Hall and Ivet Galabova. The QP solver and original language interfaces were written by Michael Feldmeier. Leona Gottwald wrote the MIP solver. The software engineering of HiGHS was developed by Ivet Galabova.

Citation

If you use HiGHS in an academic context, please acknowledge this and cite the following article

Parallelizing the dual revised simplex method, Q. Huangfu and J. A. J. Hall, *Mathematical Programming Computation*, **10** (1), 119-142, 2018. DOI: [10.1007/s12532-017-0130-5](https://doi.org/10.1007/s12532-017-0130-5)

HiGHS

https://en.wikipedia.org/wiki/HiGHS_optimization_solver

Solvers [\[edit \]](#)

Simplex [\[edit \]](#)

HiGHS has implementations of the primal and dual [revised simplex method](#) for solving LP problems, based on techniques described by Hall and McKinnon (2005),^[6] and Huangfu and Hall (2015, 2018).^{[7][8]} These include the exploitation of hyper-sparsity when solving linear systems in the simplex implementations and, for the dual simplex solver, exploitation of multi-threading. The simplex solver's performance relative to commercial and other open-source software is regularly reported using industry-standard benchmarks.^[9]

Interior point [\[edit \]](#)

HiGHS has an [interior point method](#) implementation for solving LP problems, based on techniques described by Schork and Gondzio (2020).^[10] It is notable for solving the Newton system iteratively by a [preconditioned conjugate gradient](#) method, rather than directly, via an [LDL*](#) decomposition. The interior point solver's performance relative to commercial and other open-source software is regularly reported using industry-standard benchmarks.^[11]

Mixed integer programming [\[edit \]](#)

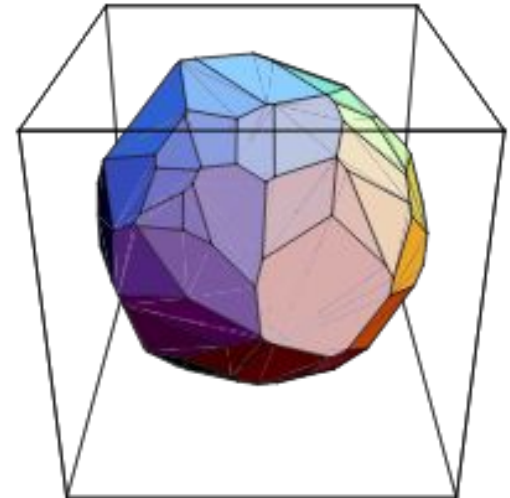
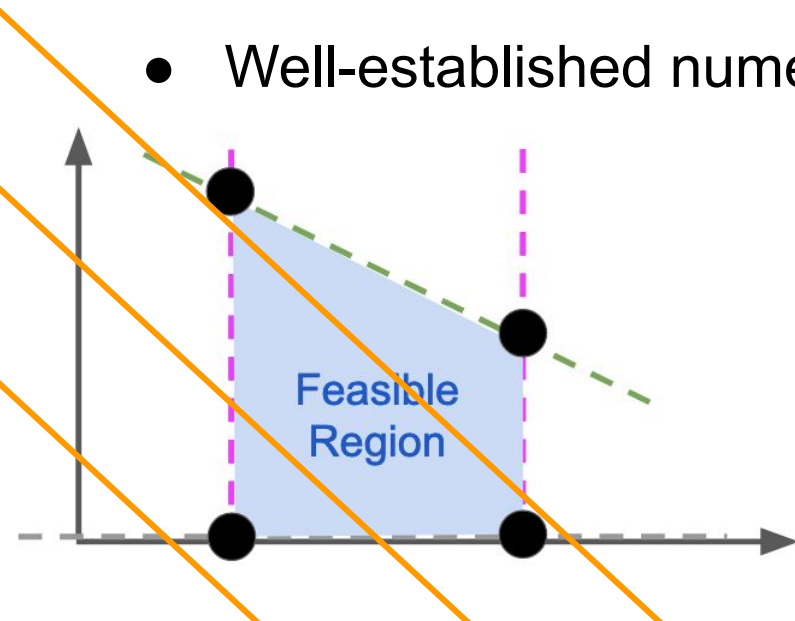
HiGHS has a branch-and-cut solver for MIP problems. Its performance relative to commercial and other open-source software is regularly reported using industry-standard benchmarks.^[12]

Quadratic programming [\[edit \]](#)

HiGHS has an active set solver for convex [quadratic programming](#) (QP) problems.

Summary

- Linear Programming (LP) is the process of minimising a linear objective function subject to linear constraints.
- Intuitively through geometry
- Computationally through the simplex method
- Widely applicable
- Well-established numerical packages



Exercises

- Convert the following LP problem into standard form

$$\text{minimize } x_1 + 5x_2 + 3x_3$$

$$\text{subject to } x_1 + 2x_2 - x_3 \leq 3,$$

$$2x_1 + 3x_2 + 2x_3 = 8,$$

$$x_2 \geq 0 \quad x_3 \geq 0,$$

$$x_1 \text{ free,}$$

- Look up the terms that are new to you in this lecture and understand them.
- Think of an LP problem that might be useful for you in real life with two design variables (so that it's in 2D). Sketch the feasible region and solve it graphically.

Thanks!

